PROYECTO FINAL BASES DE DATOS

AUTOAUDIT - SISTEMA DE AUDITORÍA AUTOMATIZADA

Integrantes:

Daniel Fernando Morales Alvizures / 5990-24-1245

Yohana Sofia Samayoa Arias / 5990-24-6796

Andy Jesús López Charuc / 1990-24-5286

Luis Enrique Chochó Portuguez / 2990-19-17704

Universidad Mariano Gálvez de Guatemala

Ingeniería en Sistemas de la Información y Ciencias de la Computación

Bases de Datos I

Ingeniero Edgar René Ornelis Hoil

Noviembre 7, 2026

Índice

1. Introducción 4
2. Objetivos 5
2.1. Objetivo General 5
2.2. Objetivos Específicos 5
3. Diagrama Entidad-Relación 6
3.1. Configuración Base del Sistema 7
3.2. Motor de Generación y Reglas 7
3.3. Contexto de Ejecución 8
3.4. Trazabilidad y Bitácora Histórica 8
4. Diccionario de Datos 9
4.1. Módulo 1: Configuración Base 9
4.2. Módulo 2: Motor de Generación y Reglas 10
4.3. Módulo 3: Contexto de Ejecución 12
4.4. Módulo 4: Trazabilidad y Bitácora Histórica 13
5. Procedimiento de Instalación y Despliegue de Triggers 15
5.1. Requisitos 15
5.2. Obtener el proyecto e instalar dependencias 16
5.3. Crear roles y base de datos 16
5.4. Configurar variables de entorno 16
5.5. Instalar el esquema e iniciar la aplicación 17
5.6. Verificación 17
5.7. Consideraciones de despliegue 17
6. Manual de Configuración y Activación de Auditoría por Tabla 18
6.1. Alcance 18
6.2. Requisitos 19
6.3. Procedimiento De Configuración Y Activación 19
6.4. Verificación Del Estado De Auditoría 25
6.5. Verificación desde el listado de tablas 25
6.6. Verificación desde la configuración individual 25
6.7. Comprobación mediante una operación sobre la tabla 26
6.8. Consideraciones Importantes 26

6.9. Solución De Problemas Frecuentes 27
7. Pruebas de Seguridad 28
7.1. Alcance y fuente 28
7.2. Evidencias funcionales relacionadas 29
7.3. Límites y medidas recomendadas 29
8. Pruebas de Validación 30
9. Manual de Usuario 35
9.1. Acceso y panel 35
9.2. Iniciar/cerrar sesión 36
9.3. Configurar la auditoría 37
9.4. Consultar y filtrar la bitácora 39
9.5. Exportar y comprender eventos 41
9.6. Flujo recomendado 41
10. Conclusiones 42

1. Introducción

AutoAudit es una aplicación web con Flask y PostgreSQL para configurar auditorías por tabla, evento y columna. Genera triggers que registran operaciones INSERT, UPDATE y DELETE en una bitácora central, incluyendo valores anteriores y nuevos y metadatos de conexión. La interfaz permite mantener configuraciones, consultar y filtrar eventos y exportar resultados CSV.

Funciones principales:

Detección de tablas y columnas mediante el diccionario de datos.

Configuración de auditoría para INSERT, UPDATE y DELETE.

- Selección de las columnas que deben ser auditadas.
- Generación dinámica de triggers.
- Un trigger de auditoría por cada tabla configurada.
- Registro de filas anteriores y nuevas.
- Registro detallado de cambios por columna.
- Registro de usuario, fecha, hora, dirección IP y estación.
- Historial de versiones de las configuraciones.
- Panel principal con el estado del sistema.
- Acceso administrativo con contraseña protegida.
- Consulta de la bitácora mediante filtros.
- Vista detallada de cada cambio.
- Exportación de reportes en formato CSV.
- Pruebas funcionales, de rendimiento y de seguridad.

2. Objetivos

2.1. Objetivo General

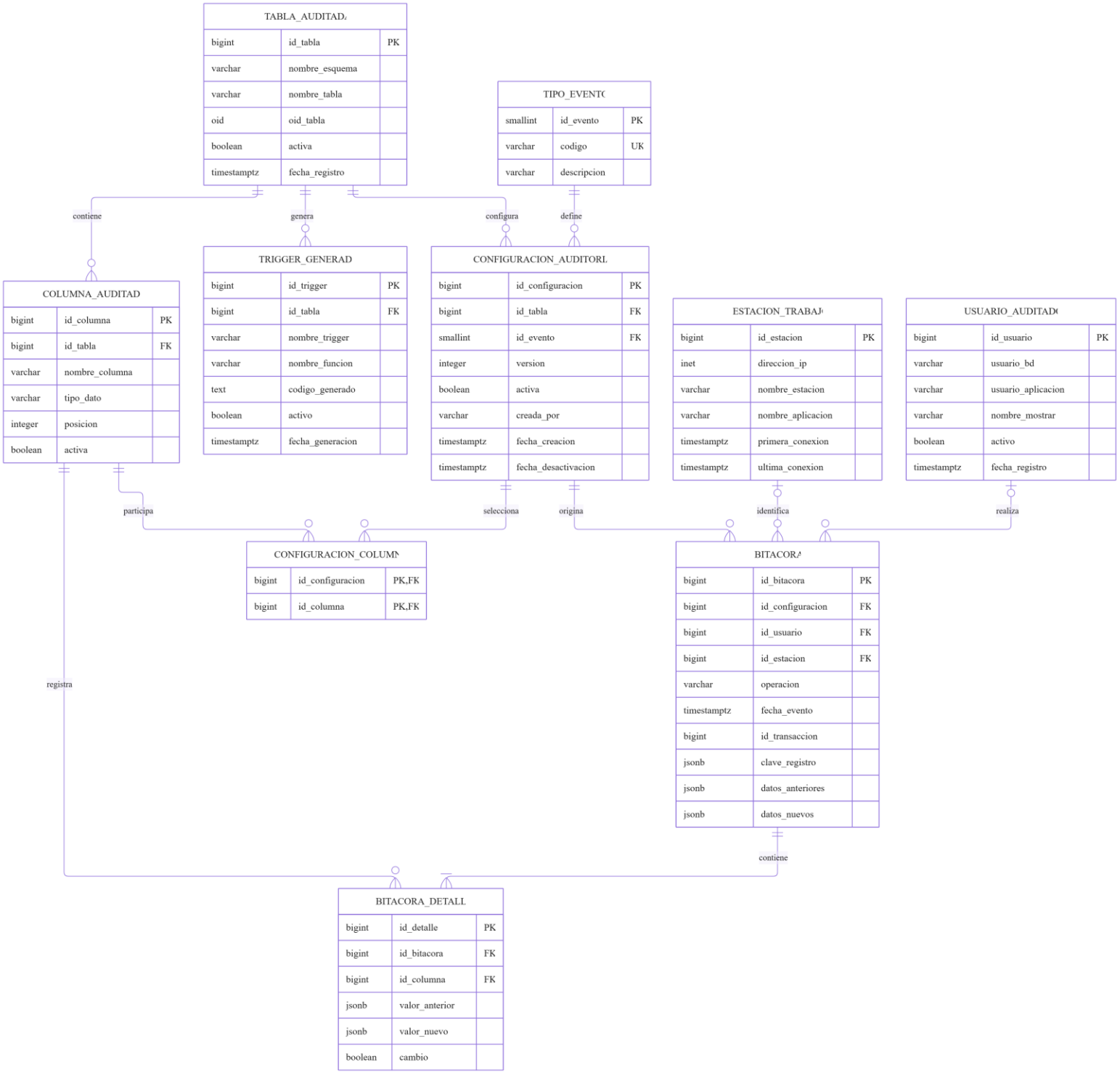
Desarrollar una solución de auditoría automática que capture cambios en tablas configurables mediante la generación dinámica de triggers, permitiendo consulta histórica de modificaciones por usuario o por columna específica.

2.2. Objetivos Específicos

- Diseñar el modelo entidad-relación completo del sistema de auditoría.
- Implementar mecanismos de configuración para selección de tablas y columnas a auditar, basado en la consulta del diccionario de datos.
- Programar la generación automática de triggers para operaciones insert, update y delete.
- Crear tablas de bitácora con información completa de cada cambio registrado.
- Desarrollar interfaces de consulta histórica por usuario o por columna afectada.

3. Diagrama Entidad-Relación

Para complementar la descripción detallada de nuestro modelo, hemos generado la siguiente representación visual del Diagrama Entidad-Relación de AutoAudit. Este diagrama ilustra gráficamente cómo interactúan las 10 entidades principales.



3.1. Configuración Base del Sistema

El punto de partida del modelo Entidad-Relación es definir exactamente qué elementos de la base de datos vamos a monitorear en un momento dado.

- **TABLA_AUDITADA:** Esta entidad funciona como el catálogo principal donde registramos cada tabla que el sistema vigilará. Almacena su identificador interno (oid_tabla), el esquema y el nombre exacto, junto con una bandera booleana (activa) para encender o apagar su monitoreo sin perder el historial de auditoría previo.

- **COLUMNA_AUDITADA:** Como la auditoría debe ser granular, esta tabla se relaciona directamente con la tabla auditada para desglosar cada uno de sus campos, guardando su tipo_dato y el orden estructural o posición.

- **TIPO_EVENTO:** Es un catálogo estático sencillo que nos permite clasificar las operaciones que disparan las alertas mediante un código único, lo que nos facilita aislar posteriormente si el cambio fue una inserción, una actualización o una eliminación.

3.2. Motor de Generación y Reglas

Una vez que sabemos qué tablas existen, el diseño contempla cómo configurar las reglas de auditoría y almacenar el código automatizado.

- **CONFIGURACION_AUDITORIA:** Es la entidad central de las políticas de nuestro sistema, ya que une la tabla auditada con el tipo de evento que queremos capturar. Le agregué un sistema de versionado (version) y fechas de vigencia (fecha_creacion, fecha_desactivacion) para mantener un registro histórico de cómo han cambiado las reglas en el tiempo.

- **CONFIGURACION_COLUMNA:** La diseñé como una tabla puente para resolver la relación de muchos a muchos, indicando con precisión qué columnas específicas participan en una versión particular de configuración de auditoría.

- **TRIGGER_GENERADO:** Aquí es donde el sistema guarda físicamente el codigo_generado en formato de texto para los disparadores dinámicos que se asocian a cada tabla, controlando su estado operativo mediante un campo booleano de activación.

3.3. Contexto de Ejecución

Para cumplir con los requerimientos de seguridad informática, no basta con saber qué dato cambió, sino quién ejecutó la acción y desde dónde se originó.

- **USUARIO_AUDITADO:** Separa la identidad de autenticación del motor de base de datos (usuario_bd) de la identidad real que la persona usó en la aplicación (usuario_aplicacion), dándonos un panorama completo del responsable.

- **ESTACION_TRABAJO:** Rastrea el origen de la red capturando la direccion_ip (utilizando el tipo de dato nativo inet), el nombre del equipo y la aplicación de conexión, registrando el rango de tiempo entre su primera y última interacción.

3.4. Trazabilidad y Bitácora Histórica

Este es el núcleo transaccional del proyecto, donde aterriza toda la información capturada por los disparadores.

- **BITACORA:** Funciona como la cabecera de nuestro historial. Cada vez que ocurre un evento, esta tabla crea un registro general que se enlaza automáticamente con la configuración que lo permitió, el usuario que lo provocó y la estación de trabajo de origen. Guarda la imagen completa de la transacción utilizando campos JSONB para separar eficientemente los datos_anteriores de los datos_nuevos.

- **BITACORA_DETALLE:** Para evitar que las consultas de reportes históricos sean lentas al analizar estructuras JSON completas, esta tabla desglosa el cambio afectado columna por columna. Compara el valor_anterior contra el valor_nuevo de manera individual y utiliza una bandera (cambio) para identificar rápidamente si ese campo específico realmente sufrió una alteración.

Este diseño asegura que el almacenamiento de la bitácora centralice los eventos sin perder el contexto técnico y humano que los provocó, logrando que el sistema completo dependa estrictamente de las reglas definidas en la base de configuración.

4. Diccionario de Datos

El siguiente diccionario de datos constituye la documentación técnica detallada de la estructura de la base de datos del proyecto. Su propósito principal es definir la sintaxis, restricciones, tipos de datos y la semántica operativa de cada una de las tablas y campos del sistema.

4.1. Módulo 1: Configuración Base

Define los objetos de la base de datos sujetos al seguimiento y los eventos que desencadenarán las capturas.

TABLA_AUDITADA Almacena el registro principal de las tablas bajo monitoreo.

Campo	Tipo de Dato	Llave	Descripción
id_tabla	bigint	PK	Identificador único de la tabla monitoreada.
nombre_esquema	varchar	-	Nombre del esquema al que pertenece la tabla.
nombre_tabla	varchar	-	Nombre físico de la tabla en la base de datos.
oid_tabla	oid	-	Identificador de objeto interno de PostgreSQL.
activa	boolean	-	Indica si el monitoreo de la tabla está encendido.
fecha_registro	timestamptz	-	Marca de tiempo exacta en que se registró la tabla.

COLUMNA_AUDITADA Detalla los campos específicos que requieren vigilancia dentro de cada tabla.

Campo	Tipo de Dato	Llave	Descripción
id_columna	bigint	PK	Identificador único del campo a auditar.
id_tabla	bigint	FK	Referencia a la tabla contenedora.
nombre_columna	varchar	-	Nombre del campo en la estructura original.
tipo_dato	varchar	-	Formato de almacenamiento (varchar, integer, etc.).
posicion	integer	-	Orden de la columna dentro de su tabla.
activa	boolean	-	Estado actual de seguimiento para esta columna.

TIPO_EVENTO Catálogo maestro de las acciones operativas.

Campo	Tipo de Dato	Llave	Descripción
id_evento	smallint	PK	Identificador interno de la acción.
codigo	varchar	UK	Acrónimo único de la operación (ej. INS, UPD, DEL).
descripcion	varchar	-	Explicación detallada del evento a nivel de sistema.

4.2. Módulo 2: Motor de Generación y Reglas

Gestiona las políticas de captura de datos y almacena el código dinámico generado para los disparadores.

CONFIGURACION_AUDITORIA Establece las reglas de monitoreo vigentes para una tabla ante un evento específico.

Campo	Tipo de Dato	Llave	Descripción
id_configuracion	bigint	PK	Identificador único de la regla de auditoría.
id_tabla	bigint	FK	Tabla a la que aplica la configuración.
id_evento	smallint	FK	Operación que activará esta regla.
version	integer	-	Número de iteración de los cambios en la regla.
activa	boolean	-	Estado de vigencia de la política actual.
creada_por	varchar	-	Usuario administrador que definió la regla.
fecha_creacion	timestamptz	-	Momento de activación de la política.
fecha_desactivacion	timestamptz	-	Momento de caducidad o reemplazo de la política.

CONFIGURACION_COLUMNA Tabla puente que asocia las políticas de auditoría con las columnas específicas a monitorear.

Campo	Tipo de Dato	Llave	Descripción
id_configuracion	bigint	PK, FK	Referencia a la regla general de auditoría.
id_columna	bigint	PK, FK	Referencia a la columna bajo vigilancia.

TRIGGER_GENERADO Almacena el código fuente de los disparadores (triggers) que Daniel desarrolló de forma automatizada.

Campo	Tipo de Dato	Llave	Descripción
id_trigger	bigint	PK	Identificador único del disparador automático.
id_tabla	bigint	FK	Tabla donde se instala físicamente el trigger.
nombre_trigger	varchar	-	Nombre técnico del objeto en la base de datos.
nombre_funcion	varchar	-	Procedimiento almacenado que ejecuta el trigger.
codigo_generado	text	-	Script completo con las instrucciones de captura.
activo	boolean	-	Estado operativo del trigger.
fecha_generacion	timestamptz	-	Momento en que el código fue compilado.

4.3. Módulo 3: Contexto de Ejecución

Identifica el origen técnico de los cambios registrados.

USUARIO_AUDITADO Catálogo de los perfiles que interactúan con la información sensible.

Campo	Tipo de Dato	Llave	Descripción
id_usuario	bigint	PK	Identificador único del usuario.
usuario_bd	varchar	-	Credencial utilizada para conectarse al motor.
usuario_aplicacion	varchar	-	Credencial humana desde el frontend o sistema.

Campo	Tipo de Dato	Llave	Descripción
nombre_mostrar	varchar	-	Nombre completo y legible de la persona.
activo	boolean	-	Estado de la cuenta de usuario.
fecha_registro	timestamptz	-	Fecha de primera detección en el sistema.

ESTACION_TRABAJO Registro de los dispositivos y programas desde donde se originan las alteraciones de datos.

Campo	Tipo de Dato	Llave	Descripción
id_estacion	bigint	PK	Identificador único del equipo de acceso.
direccion_ip	inet	-	Dirección de red desde donde se ejecutó la acción.
nombre_estacion	varchar	-	Nombre del host o computadora del usuario.
nombre_aplicacion	varchar	-	Software utilizado (ej. DBeaver, Sistema Web).
primera_conexion	timestamptz	-	Fecha del primer acceso detectado.
ultima_conexion	timestamptz	-	Fecha del registro más reciente desde el equipo.

4.4. Módulo 4: Trazabilidad y Bitácora Histórica

Contiene los registros definitivos que documentan las alteraciones de los datos en el tiempo.

BITACORA Cabecera transaccional que documenta el evento general capturado por el trigger.

Campo	Tipo de Dato	Llave	Descripción
id_bitacora	bigint	PK	Identificador principal del evento histórico.
id_configuracion	bigint	FK	Regla que ordenó la captura del evento.
id_usuario	bigint	FK	Responsable de ejecutar la transacción.
id_estacion	bigint	FK	Equipo desde donde se originó el cambio.
operacion	varchar	-	Acción literal ejecutada (Insert, Update, Delete).
fecha_evento	timestamptz	-	Momento exacto del impacto en la base de datos.
id_transaccion	bigint	-	ID de la transacción global en PostgreSQL.
clave_registro	jsonb	-	Valores de la llave primaria del registro afectado.
datos_anteriores	jsonb	-	Fotografía completa de la fila antes del cambio.
datos_nuevos	jsonb	-	Fotografía completa de la fila después del cambio.

BITACORA_DETALLE Desglose granular que compara valor por valor únicamente en los campos que sufrieron modificaciones.

Campo	Tipo de Dato	Llave	Descripción
id_detalle	bigint	PK	Identificador del cambio específico a nivel columna.
id_bitacora	bigint	FK	Referencia al evento maestro.
id_columna	bigint	FK	Columna puntual que sufrió la alteración.
valor_anterior	jsonb	-	Dato original previo a la actualización.
valor_nuevo	jsonb	-	Dato resultante almacenado tras el evento.
cambio	boolean	-	Bandera que confirma si hubo una mutación real.

5. Procedimiento de Instalación y Despliegue de Triggers

Las instrucciones de instalación están orientadas al entorno Ubuntu indicado por el proyecto. Los resultados de prueba de este documento se transcriben de la documentación del repositorio; no se ejecutaron nuevamente como parte de la generación del Word.

5.1. Requisitos

- Ubuntu 26.04 LTS, PostgreSQL 18, Python 3, Git y acceso al repositorio.
- Conectividad local a PostgreSQL y permisos administrativos para crear roles/base de datos.
- El proyecto reporta un ambiente de prueba con 2 CPU y aproximadamente 3.3 GB de RAM; es referencia, no un mínimo medido de producción.

5.2. Obtener el proyecto e instalar dependencias

```
git clone https://github.com/DanielM73/autoaudit.git
cd autoaudit
sudo apt update
sudo apt install git python3-pip python3-venv postgresql-client
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Si el repositorio es privado, la persona que instala necesita permisos de acceso. El repositorio documenta también la clonación por SSH.

5.3. Crear roles y base de datos

El diseño separa al propietario de objetos (autoaudit_owner) del usuario de conexión web (autoaudit_app). Use contraseñas robustas y únicas; no las escriba en el repositorio ni reutilice las credenciales de administrador.

```
sudo -u postgres psql
CREATE ROLE autoaudit_owner WITH LOGIN;
\password autoaudit_owner
CREATE ROLE autoaudit_app WITH LOGIN;
\password autoaudit_app
CREATE DATABASE autoaudit_db WITH OWNER = autoaudit_owner ENCODING = 'UTF8' TEMPLATE = template0;
REVOKE ALL ON DATABASE autoaudit_db FROM PUBLIC;
GRANT CONNECT ON DATABASE autoaudit_db TO autoaudit_app;
\c autoaudit_db
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO autoaudit_app;
\q
```

Las contraseñas se solicitan de forma interactiva con \password. Aplique los permisos de esquema indicados por el manual del proyecto; evite otorgar privilegios amplios al rol de aplicación.

5.4. Configurar variables de entorno

```
cp .env.example .env
nano .env
FLASK_APP=run.py
FLASK_DEBUG=0
SECRET_KEY=<clave aleatoria segura>
ADMIN_USERNAME=admin
ADMIN_PASSWORD_HASH=<hash generado con Werkzeug>
DB_HOST=127.0.0.1
DB_PORT=5432
DB_NAME=autoaudit_db
DB_USER=autoaudit_app
DB_PASSWORD=<contraseña de autoaudit_app>
```

Genere una clave de sesión aleatoria localmente y el hash de contraseña con generate_password_hash de Werkzeug (el manual reporta scrypt). Inicie sesión con la contraseña original, nunca con el hash. El archivo .env debe permanecer privado y fuera del control de versiones; el ejemplo del repositorio tiene FLASK_DEBUG=1, por lo que cámbielo a 0 antes de iniciar cualquier instancia compartida.

5.5. Instalar el esquema e iniciar la aplicación

```
chmod +x scripts/instalar_base_datos.sh
./scripts/instalar_base_datos.sh
chmod +x scripts/iniciar_aplicacion.sh
./scripts/iniciar_aplicacion.sh
```

El instalador ejecuta los DDL, las funciones SQL, el diccionario de datos y los datos de demostración en secuencia como autoaudit_owner. A continuación, el script de inicio activa .venv y ejecuta run.py. La aplicación escucha en el puerto 5000 (o PORT); el código enlaza a 0.0.0.0. Deténgala con Ctrl+C.

5.6. Verificación

- Abra http://127.0.0.1:5000/health. Se espera estado ok y datos de la conexión PostgreSQL; ante fallo de base de datos la aplicación responde 503.

- Abra http://127.0.0.1:5000 e inicie sesión con ADMIN_USERNAME y la contraseña original.

- En Configuración, compruebe que se detectan las tablas y que los triggers/operaciones deseados aparecen activos.

- Realice una operación de prueba y confirme el evento en Bitácora antes de dar por terminada la instalación.

5.7. Consideraciones de despliegue

La ejecución incluida (python run.py / servidor de desarrollo Flask) sirve para desarrollo o demostraciones controladas; el proyecto no proporciona una configuración de servidor WSGI, proxy inverso, HTTPS ni servicio de sistema. No exponga directamente el servidor de desarrollo a Internet. Para producción, planifique y pruebe, como tareas externas al repositorio: servidor WSGI de producción, proxy HTTPS con certificados, firewall y segmentación de red, gestión protegida de secretos, registro/monitorización, copias de seguridad y procedimiento de restauración.

El endpoint /health no requiere autenticación y devuelve metadatos de la base de datos y de PostgreSQL; limite su alcance de red o proteja el acceso en el proxy si el entorno lo requiere. Configure la cookie Secure al operar exclusivamente sobre HTTPS (no está activada en la configuración visible de la aplicación).

6. Manual de Configuración y Activación de Auditoría por Tabla

Sistema: AutoAudit
Motor de base de datos: PostgreSQL 18
Aplicación: Flask / Python
Tipo de documento: Manual técnico de configuración

El presente manual tiene como finalidad establecer el procedimiento y los pasos necesarios para realizar la configuración y activación de auditoría por tabla en AutoAudit, permitiendo definir los eventos y las columnas que serán supervisados, así como verificar que los mecanismos de auditoría se encuentren correctamente implementados.

6.1. Alcance

Este manual comprende los procedimientos relacionados con:

- Identificación de las tablas disponibles para auditoría.
- Selección de la tabla que se desea supervisar.
- Configuración de los eventos de auditoría.
- Selección de las columnas que serán auditadas.
- Activación de la configuración correspondiente.
- Generación automática del trigger de auditoría.
- Actualización de configuraciones existentes.
- Desactivación individual de eventos de auditoría.
- Verificación del estado de la configuración.

El procedimiento está dirigido al administrador autorizado del sistema AutoAudit y presupone que la aplicación, la base de datos y sus componentes de auditoría se encuentran previamente instalados y funcionando.

6.2. Requisitos

Antes de realizar la configuración de auditoría por tabla, es necesario verificar el cumplimiento de los siguientes requisitos:

- Disponer de una instalación funcional de PostgreSQL 18.
- Contar con la aplicación AutoAudit instalada y configurada.
- Tener disponibles las estructuras, funciones y procedimientos almacenados del esquema auditoria.
- Verificar que la conexión entre AutoAudit y PostgreSQL se encuentre activa.
- Disponer de credenciales administrativas válidas para ingresar a la aplicación.
- Confirmar que las tablas que se desean auditar existan en la base de datos y sean compatibles con el mecanismo de auditoría.
- Verificar que el propietario de las tablas cuente con los permisos necesarios para generar triggers.

Consideración: AutoAudit permite auditar tablas ordinarias y particionadas. No admite la configuración de los esquemas internos auditoria, information_schema ni aquellos cuyo nombre comienza con pg_.

6.3. Procedimiento De Configuración Y Activación

1. **Ingreso al sistema AutoAudit:**

Para comenzar el procedimiento, se debe acceder a la aplicación AutoAudit mediante un navegador web, utilizando la dirección local o la dirección del servidor donde se encuentre ejecutándose el sistema.

Una vez cargada la aplicación, se debe ingresar mediante las credenciales administrativas previamente configuradas.

El acceso administrativo permite gestionar las configuraciones de auditoría y realizar modificaciones sobre los eventos y las columnas que serán supervisados.

2. **Acceso al módulo de configuración**

Después de iniciar sesión, se debe ingresar al apartado **Configuración de auditoría**.

Este módulo permite visualizar las tablas disponibles en la base de datos y consultar el estado de configuración de cada una.

La información presentada incluye:

3. **Esquema:** identifica el esquema al que pertenece la tabla.
4. **Tabla:** indica el nombre de la tabla disponible.
5. **Eventos configurados:** muestra los eventos de auditoría que se encuentran configurados.
6. **Trigger:** indica si el trigger correspondiente se encuentra instalado.
7. **Acción:** permite acceder a la configuración individual de cada tabla.

Las tablas se presentan identificadas mediante su esquema y nombre, permitiendo diferenciarlas cuando existen tablas con nombres iguales en distintos esquemas.

1) Selección de la tabla que se desea auditar

Dentro del listado de tablas disponibles, se debe identificar aquella sobre la cual se desea implementar la auditoría.

Una vez localizada, se debe seleccionar la opción **Configurar**.

El sistema mostrará una sección individual de configuración para la tabla seleccionada.

En esta sección se identifica el esquema y el nombre de la tabla, además de presentar las columnas que la componen.

La información de las columnas incluye:

- Nombre de la columna.
- Tipo de dato.
- Identificación de clave primaria, cuando corresponda.
- Indicador de obligatoriedad cuando la columna no permite valores nulos.

La identificación de las columnas permite determinar cuáles serán consideradas dentro del proceso de auditoría.

2) Configuración de los eventos de auditoría

AutoAudit permite configurar individualmente tres tipos de eventos:

- **INSERT:** Permite registrar las operaciones de inserción de nuevos registros en la tabla.

Cuando se produce una inserción, el sistema almacena la información correspondiente al nuevo registro, junto con los datos de identificación y contexto disponibles para la auditoría.

- **UPDATE:** Permite registrar las modificaciones realizadas sobre los registros existentes.

El sistema compara los valores anteriores y nuevos de las columnas seleccionadas, identificando si se produjo alguna modificación que deba quedar registrada.

Cuando ninguno de los valores de las columnas seleccionadas presenta cambios, no se genera un registro de auditoría para esa operación.

- **DELETE:** Permite registrar las operaciones de eliminación de registros.

En este caso, se conserva la información del registro antes de su eliminación, permitiendo mantener evidencia de los datos que fueron eliminados.

Cada evento cuenta con su propia sección de configuración, por lo que es posible habilitar INSERT, UPDATE y DELETE de manera independiente.

3) Selección de las columnas que serán auditadas

Para cada evento se presenta el listado de columnas pertenecientes a la tabla seleccionada.

El administrador puede establecer qué columnas deben considerarse durante el proceso de auditoría mediante las casillas de selección.

El sistema proporciona dos opciones para facilitar esta configuración:

- **Seleccionar todas:** marca todas las columnas disponibles de la tabla.
- **Quitar selección:** desmarca todas las columnas seleccionadas.

Existen dos modalidades de configuración:

a) Auditoría de columnas específicas

Se seleccionan únicamente las columnas cuyos cambios se desean supervisar.

Esta modalidad permite establecer un control más específico sobre la información que se desea identificar en los registros detallados de auditoría.

b) Auditoría de todas las columnas

Cuando no se selecciona ninguna columna, AutoAudit interpreta la configuración como una solicitud para auditar todas las columnas disponibles de la tabla.

De esta manera, no es necesario seleccionar manualmente cada columna cuando se requiere una supervisión completa.

Importante: la selección de columnas se realiza de forma independiente para cada evento. Por ejemplo, una tabla puede tener configuradas determinadas columnas para INSERT y un conjunto diferente para UPDATE.

Adicionalmente, el sistema conserva los datos anteriores y nuevos de la fila en formato JSONB en el registro principal de la bitácora. La selección de columnas determina los campos considerados en el detalle de auditoría y en la detección de cambios para UPDATE.

4) Activación del evento de auditoría

Una vez seleccionado el evento y definidas las columnas correspondientes, se debe presionar el botón de activación que aparece al finalizar su sección.

El botón puede presentar los siguientes nombres:

- **Activar INSERT**
- **Activar UPDATE**
- **Activar DELETE**

Al ejecutar esta acción, AutoAudit realiza internamente el siguiente procedimiento:

1) Valida que la tabla seleccionada exista y esté disponible para auditoría.
2) Comprueba que el evento solicitado corresponda a uno de los tipos permitidos.
3) Verifica que las columnas seleccionadas pertenezcan a la tabla.
4) Ejecuta la función almacenada auditoria.configurar_auditoria().
5) Registra la configuración correspondiente al evento seleccionado.

6) Genera una nueva versión de la configuración.
7) Solicita la generación o actualización del trigger mediante la función auditoria.generar_trigger().
8) Confirma los cambios en la base de datos.

Cuando la operación finaliza correctamente, el sistema presenta un mensaje de confirmación indicando el evento configurado, el identificador de configuración y el nombre del trigger generado.

La configuración queda almacenada en PostgreSQL y puede consultarse posteriormente desde el módulo de configuración.

5) Generación automática del trigger

Una vez registrada la configuración, AutoAudit genera automáticamente el trigger encargado de detectar las operaciones realizadas sobre la tabla.

Para ello utiliza la función:

```
auditoria.generar_trigger()
```

Esta función genera un trigger asociado específicamente a la tabla configurada.

El trigger se ejecuta después de las operaciones INSERT, UPDATE y DELETE, utilizando la función:

```
auditoria.registrar_cambio()
```

Esta última consulta las configuraciones activas y determina si el evento ejecutado debe generar un registro de auditoría.

De esta manera, aunque el trigger se encuentre preparado para detectar los tres tipos de operaciones, únicamente se registran aquellas que estén habilitadas en la configuración correspondiente.

Cada tabla configurada cuenta con su propio trigger de auditoría, lo que permite administrar de manera independiente el seguimiento de las operaciones realizadas sobre diferentes tablas.

6) Actualización de una configuración existente

Cuando una tabla ya cuenta con un evento configurado, el sistema permite modificar las columnas que serán consideradas durante la auditoría.

Para realizar esta operación se deben seguir los siguientes pasos:

1) Ingresar al módulo de Configuración de auditoría.
2) Localizar la tabla que se desea modificar.
3) Seleccionar la opción Configurar.
4) Identificar el evento que se encuentra previamente configurado.
5) Modificar la selección de columnas según los nuevos requerimientos.
6) Presionar el botón **Actualizar**, correspondiente al evento.
7) Esperar el mensaje de confirmación del sistema.

Al guardar una modificación, AutoAudit desactiva la versión anterior de la configuración y genera una nueva versión activa.

El historial de configuraciones permite conservar las versiones anteriores y mantener la trazabilidad de los cambios administrativos realizados.

Posteriormente, el sistema actualiza el trigger correspondiente para mantener la configuración vigente.

7) Desactivación de un evento de auditoría

AutoAudit también permite desactivar individualmente un evento que se encuentre configurado.

Para realizar esta operación:

1) Ingresar al módulo de Configuración de auditoría.
2) Seleccionar la tabla correspondiente.
3) Identificar el evento que se desea desactivar.
4) Presionar el botón **Desactivar**, correspondiente al evento.
5) Confirmar la acción cuando el sistema solicite autorización.
6) Verificar el mensaje de respuesta presentado por la aplicación.

Al ejecutar esta operación, el sistema modifica el estado de la configuración correspondiente para que deje de considerarse activa.

Consideración técnica: la implementación actual requiere que permanezca al menos una configuración activa para que la función generadora pueda mantener el trigger de la tabla. Por esta razón, la

desactivación del último evento activo no se completa correctamente desde la interfaz. Para evitar errores, se debe conservar al menos un evento activo mientras se utiliza esta funcionalidad.

6.4. Verificación Del Estado De Auditoría

Después de completar la configuración, es necesario comprobar que el evento seleccionado se encuentre correctamente registrado y que el trigger correspondiente esté instalado.

La verificación puede realizarse desde dos apartados del sistema.

6.5. Verificación desde el listado de tablas

Al regresar al módulo de Configuración de auditoría, se debe localizar nuevamente la tabla configurada.

El sistema presenta el estado de los eventos configurados y la condición del trigger.

Los posibles indicadores son:

- **Sin configurar:** la tabla no presenta eventos de auditoría activos.
- **Eventos configurados:** muestra los eventos que cuentan con configuraciones activas.
- **Trigger activo:** indica que el trigger correspondiente se encuentra instalado en PostgreSQL.
- **Trigger no instalado:** indica que no se ha detectado un trigger instalado para la tabla.

6.6. Verificación desde la configuración individual

Al ingresar nuevamente a la configuración de una tabla, se puede consultar:

- Los eventos que se encuentran configurados.
- La versión vigente de cada configuración.
- El nombre del trigger generado.
- La fecha de generación del trigger.
- El estado de instalación del trigger.

Esta información permite confirmar que la configuración se encuentra registrada y que el mecanismo de auditoría está disponible para detectar las operaciones correspondientes.

6.7. Comprobación mediante una operación sobre la tabla

Para comprobar el funcionamiento de la auditoría, se puede realizar una operación controlada sobre la tabla configurada, siempre que se disponga de autorización para modificar sus datos.

La operación debe corresponder a alguno de los eventos habilitados.

Por ejemplo:

- Si se encuentra habilitado INSERT, se debe insertar un nuevo registro.
- Si se encuentra habilitado UPDATE, se debe modificar el valor de alguna de las columnas seleccionadas.
- Si se encuentra habilitado DELETE, se debe eliminar un registro de prueba.

Posteriormente, se puede consultar la bitácora para verificar que la operación haya sido registrada correctamente.

La información almacenada puede incluir el tipo de operación, la tabla afectada, los valores anteriores y nuevos, el usuario de base de datos, la fecha y hora, así como los datos de estación y dirección IP cuando se encuentren disponibles.

6.8. Consideraciones Importantes

Para garantizar el funcionamiento adecuado de AutoAudit, se deben tomar en cuenta las siguientes consideraciones:

1) La configuración se realiza individualmente para cada tabla.
2) Los eventos INSERT, UPDATE y DELETE pueden habilitarse de manera independiente.
3) La selección de columnas también es independiente para cada evento.
4) Si no se selecciona ninguna columna, el sistema interpreta que se deben considerar todas las columnas disponibles.
5) Al actualizar una configuración, se genera una nueva versión y la anterior deja de estar activa.
6) La generación del trigger se realiza automáticamente después de guardar una configuración.
7) El sistema no permite configurar tablas pertenecientes a los esquemas internos excluidos.
8) Para generar el trigger, el propietario de la función debe contar con los permisos necesarios sobre la tabla.
9) La desactivación de eventos debe realizarse considerando que la implementación requiere mantener al menos una configuración activa por tabla.

10) Los registros históricos de auditoría se conservan en la bitácora centralizada y no deben confundirse con el estado actual de la configuración.

6.9. Solución De Problemas Frecuentes

→ **Problema: La tabla que se desea auditar no aparece en el listado.**

Posibles causas:

- La tabla no existe en la base de datos.
- Pertenece a un esquema excluido.
- No corresponde a una tabla ordinaria o particionada.
- La estructura de auditoría no está actualizada.

Acción recomendada: verificar la existencia de la tabla, su esquema y los permisos de acceso correspondientes.

→ **Problema: No se puede guardar la configuración.**

Posibles causas:

- Se seleccionó un evento no válido.
- Se recibió una columna que no pertenece a la tabla.
- Existe un problema de conexión con PostgreSQL.
- La función de configuración presentó un error.

Acción recomendada: revisar la selección realizada, comprobar la conexión y consultar el mensaje de error presentado por el sistema.

→ **Problema: No se pudo generar el trigger.**

Posibles causas:

- La tabla no cuenta con configuraciones activas.
- El propietario de la función no tiene permisos para crear triggers sobre la tabla.
- Se produjo un error durante la ejecución de la función generadora.

Acción recomendada: verificar que exista al menos una configuración activa y que los permisos sobre la tabla sean correctos.

→ **Problema: Se realizaron modificaciones, pero no aparecen en la bitácora.**

Posibles causas:

- El evento ejecutado no se encuentra habilitado.
- La modificación no afectó ninguna de las columnas seleccionadas para UPDATE.
- El trigger no se encuentra instalado.
- La operación no se completó correctamente.

Acción recomendada: revisar la configuración del evento, comprobar el estado del trigger y verificar nuevamente la operación realizada.

7. Pruebas de Seguridad

7.1. Alcance y fuente

Los resultados de esta sección se basan en docs/pruebas-realizadas.md y database/tests/005_prueba_seguridad.sql. Se documentan como resultados reportados por el proyecto; no se repitieron en este entorno. La prueba SQL comprueba privilegios del rol conectado y espera que el script se ejecute como autoaudit_app. No constituye una prueba de penetración, revisión criptográfica ni garantía de ausencia de vulnerabilidades.

Control probado/documentado	Resultado registrado	Alcance
Conexión a base de datos y uso del esquema auditoria	Correcto	Permisos necesarios para el rol de aplicación.
Lectura de bitácora	Correcto	El rol de aplicación puede consultar los registros.
INSERT, UPDATE y DELETE directos sobre bitácora	Rechazados	El SQL prueba escritura directa no autorizada; no cubre acceso del propietario/superusuario.
Crear objetos en public	Rechazado	Verificación de privilegio CREATE.

Ejecución directa de auditoria.registrar_cambio()	Rechazada	La función interna del trigger no se concede al rol de aplicación.
Autenticación y rutas administrativas	Correcto según reporte	Prueba funcional documentada: acceso restringido sin sesión.
CSRF en formularios de login, configuración y logout	Implementado y reportado correcto	Token de sesión comparado en acciones POST; el reporte indica cierre protegido.
Contraseña administrativa	Hash scrypt documentado	La aplicación valida el hash; la contraseña original se gestiona fuera del código.
Cookies de sesión	HTTPOnly y SameSite=Lax; duración 2 horas	Secure no aparece activado en la configuración revisada.

7.2. Evidencias funcionales relacionadas

El registro de pruebas también informa resultados correctos para el endpoint de salud, configuración de triggers y bitácora, filtros y exportación. Las operaciones de inserción, actualización y eliminación quedaron reflejadas en la bitácora. La prueba de rendimiento reporta 250 actualizaciones en 158.26 ms (promedio 0.6330 ms), 250 registros y 500 detalles; se ejecutó dentro de una transacción con ROLLBACK. Estas cifras corresponden al equipo de pruebas documentado y no deben interpretarse como garantía de rendimiento en otros entornos.

7.3. Límites y medidas recomendadas

- No se documentan pruebas de penetración, fuzzing, análisis de dependencias, escaneo automatizado de vulnerabilidades, abuso de fuerza bruta ni validación de exposición a Internet.
- No se observa limitación de intentos de inicio de sesión. Añada control de tasa/retardo y alertas si el sistema se despliega en una red compartida o pública.
- La configuración de Flask revisada activa HttpOnly y SameSite, pero no Secure. Use HTTPS y configure la cookie Secure en el entorno de producción.
- El archivo .env.example indica FLASK_DEBUG=1; el valor de despliegue debe ser 0. No ejecute modo debug frente a usuarios no confiables.

- La bitácora es de solo lectura para autoaudit_app, pero no es inmutable frente al propietario de la base de datos o un superusuario. Proteja credenciales privilegiadas, copias y accesos administrativos; evalúe almacenamiento externo/inmutable si el requisito lo exige.
- Limite el acceso de red a Flask, PostgreSQL y /health; configure TLS, firewall, actualizaciones, copias de seguridad y una restauración probada.
- Verifique que los datos auditados no incluyan secretos o información personal innecesaria. Los valores antiguos/nuevos y el CSV pueden contener datos sensibles.

8. Pruebas de Validación

Operación INSERT (con registro en bitacora)

- ***Objetivo:*** Comprobar que la inserción de un nuevo registro en una tabla bajo auditoría detona el trigger dinámico y almacena correctamente el evento inicial en la bitácora central.
- **Condicionantes:** La tabla origen (ej. producto) debe estar registrada en TABLA_AUDITADA y tener el evento INSERT habilitado en las reglas de configuración.

Pasos de Ejecución:

- Ejecutar el script SQL de prueba (002_prueba_insert.sql) para insertar un nuevo artículo con valores iniciaes.
- Consultar la tabla auditoria.bitacora y auditoria.bitacora_detalle para validar la captura del evento.

Validación:

- La cabecera de la bitácora refleja la operación INSERT.
- El campo datos_anteriores se registra como nulo, confirmando que no existía estado previo.
- El campo datos_nuevos contiene la estructura exacta del registro recién insertado.

Evidencia: Preparación del script de prueba para la operación INSERT.

```
 GNU nano 8.7.1                                                database/tes
ORDER BY bitacora.id_bitacora DESC
LIMIT 1;

SELECT
    columna.nombre_columna,
    detalle.valor_anterior,
    detalle.valor_nuevo,
    detalle.cambio
FROM auditoria.bitacora_detalle AS detalle
JOIN auditoria.columna_auditada AS columna
    ON columna.id_columna = detalle.id_columna
WHERE detalle.id_bitacora = (
    SELECT max(bitacora.id_bitacora)
    FROM auditoria.bitacora AS bitacora
    JOIN auditoria.configuracion_auditoria AS configuracion
        ON configuracion.id_configuracion =
           bitacora.id_configuracion
    JOIN auditoria.tabla_auditada AS tabla
        ON tabla.id_tabla = configuracion.id_tabla
    WHERE tabla.nombre_esquema = 'demo'
      AND tabla.nombre_tabla = 'producto'
      AND bitacora.operacion = 'INSERT'
)

^G Help         ^O Write Out    ^F Where Is     ^K Cut          ^T Execute      ^C
^X Exit         ^R Read File    ^\ Replace      ^U Paste        ^J Justify      ^/
```

Resultado en consola evidenciando la creación del producto "Audifonos USB" y el registro detallado en la bitácora con el flag de cambio en true.

```
(1 row)

 id_producto |    nombre     | precio | existencia
-------------+---------------+--------+------------
           5 | Audifonos USB | 225.00 |         15
(1 row)

INSERT 0 1
COMMIT
 id_bitacora | operacion |   clave_registro   | sin_datos_anteriores | nombre_nuevo  | precio_nuevo | existencia_nueva |
p
-------------+-----------+--------------------+----------------------+---------------+--------------+------------------+
--
           2 | INSERT    | {"id_producto": 5} | t                    | Audifonos USB | 225.00       | 15               |
(1 row)

   nombre_columna    | valor_anterior |            valor_nuevo             | cambio
---------------------+----------------+------------------------------------+--------
 id_producto         |                | 5                                  | t
 nombre              |                | "Audifonos USB"                    | t
 precio              |                | 225.00                             | t
 existencia          |                | 15                                 | t
 activo              |                | true                               | t
 fecha_actualizacion |                | "2026-10-29T02:29:42.162888+00:00" | t
(6 rows)
```

Operación UPDATE

- ***Objetivo***: Validar que una actualización de datos capture de forma exacta la diferencia entre la versión previa y la nueva versión del registro modificado, documentando solo los campos alterados.
- **Condicionantes:** El registro de prueba debe existir previamente en la base de datos y la regla para el evento UPDATE debe estar activa.

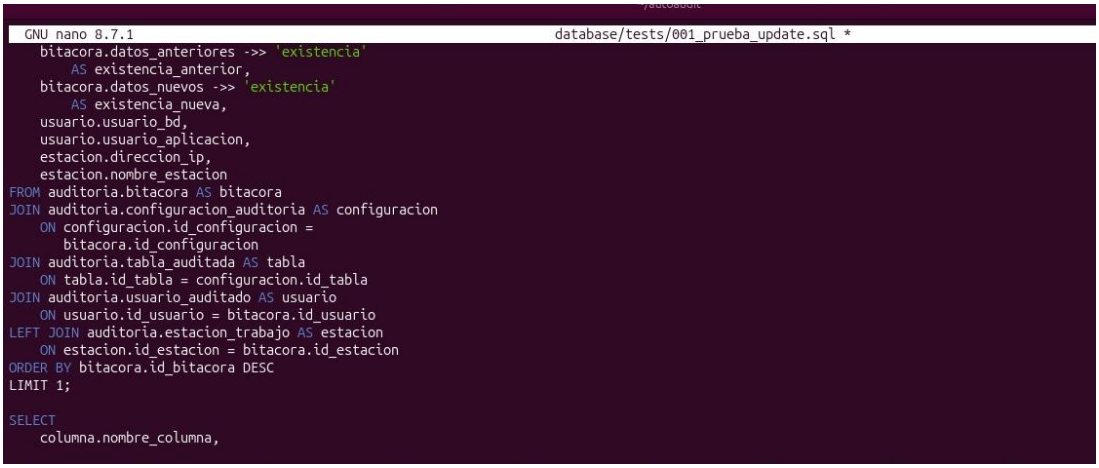
Pasos de Ejecución:

- Ejecutar el script 001_prueba_update.sql modificando valores específicos (ej. alterando el precio y la cantidad de existencias).
- Extraer el registro generado en la cabecera de auditoría cruzando los datos con la tabla de usuarios y detalle de columnas.

Validación:

- El detalle muestra únicamente las columnas alteradas con su valor_anterior y valor_nuev.
- El campo datos_anteriores almacena la fotografía exacta del registro antes del cambio

Evidencia: Preparación del script de prueba para la operación UPDATE



Resultado en consola validando el registro del usuario y la comparación exacta de precios y existencias antes y después de la actualización.

```
 nombre_esquema | nombre_tabla | operacion |   clave_registro   | precio_anterior | precio_nuevo | existencia_anterior | existencia_nueva |  usuario_bd   |
acion | direccion_ip | nombre_estacion
----------------+--------------+-----------+--------------------+-----------------+--------------+---------------------+------------------+---------------+
------+--------------+----------------
 demo           | producto     | UPDATE    | {"id_producto": 2} | 175.50          | 185.50       | 35                  | 34               | autoaudit_app |
      | 10.0.2.15    | VM Ubuntu BD1
```

Operación DELETE (preservacion de datos en bitacora)

- ***Objetivo***: Asegurar que, al eliminar permanentemente un registro de la base de datos principal, su información histórica y su último estado conocido queden resguardados de forma íntegra.
- **Condicionantes:** El registro a eliminar debe existir y la política de auditoría para el evento DELETE debe estar parametrizada.

Pasos de Ejecución:

- Ejecutar el script 003_prueba_delete.sql para borrar el registro específico de la tabla.
- Validar que el registro ya no existe en la tabla de origen y consultar la tabla auditoria.bitacora buscando la llave primaria destruida.

Validación:

- La bitácora captura el evento como DELETE
- El campo datos_anteriores preserva de forma íntegra el JSON completo del registro justo antes de su destrucción
- El campo datos_nuevos aparece como nulo o vacío.

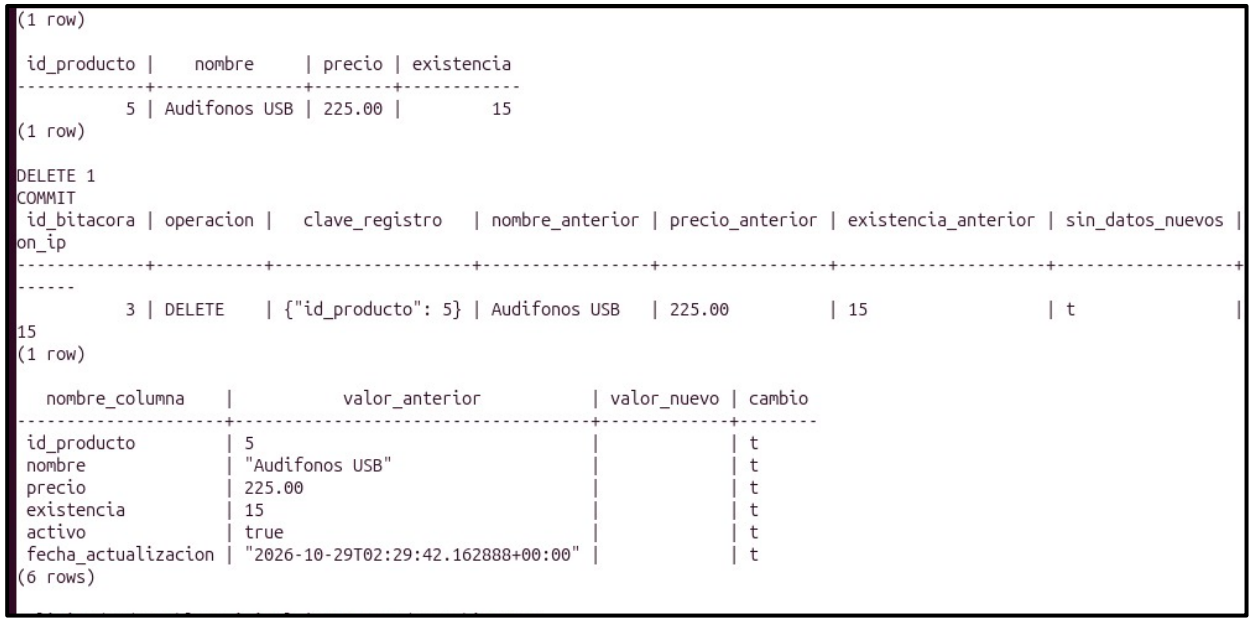
Evidencia: Preparación del script de prueba para la operación DELETE verificando la inexistencia del registro en origen.

```
 GNU nano 8.7.1                                  database/tests/003_prueba_delete.sql *
    JOIN auditoria.configuracion_auditoria AS configuracion
        ON configuracion.id_configuracion =
           bitacora.id_configuracion
    JOIN auditoria.tabla_auditada AS tabla
        ON tabla.id_tabla = configuracion.id_tabla
    WHERE tabla.nombre_esquema = 'demo'
      AND tabla.nombre_tabla = 'producto'
      AND bitacora.operacion = 'DELETE'
)
ORDER BY columna.posicion;

SELECT
    NOT EXISTS (
        SELECT 1
        FROM demo.producto
        WHERE nombre = 'Audifonos USB'
    ) AS eliminado_de_tabla_original,
    EXISTS (
        SELECT 1
        FROM auditoria.bitacora
        WHERE operacion = 'DELETE'
          AND datos_anteriores ->> 'nombre' =
              'Audifonos USB'

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute   ^C Location  M-U Undo    M-A Set Mark  M-] To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo    M-6 Copy      ^B Where Was
```

Resultado en consola confirmando la remoción del registro original y la persistencia segura de sus datos en la columna 'datos_anteriores' de la bitácora.



```
(1 row)

 id_producto |    nombre     | precio | existencia
-------------+---------------+--------+------------
           5 | Audifonos USB | 225.00 |         15
(1 row)

DELETE 1
COMMIT
 id_bitacora | operacion |   clave_registro   | nombre_anterior | precio_anterior | existencia_anterior | sin_datos_nuevos |
on_ip
-------------+-----------+--------------------+-----------------+-----------------+---------------------+------------------+
------
           3 | DELETE    | {"id_producto": 5} | Audifonos USB   | 225.00          | 15                  | t                |
15
(1 row)

   nombre_columna    |           valor_anterior           | valor_nuevo | cambio
---------------------+------------------------------------+-------------+--------
 id_producto         | 5                                  |             | t
 nombre              | "Audifonos USB"                    |             | t
 precio              | 225.00                             |             | t
 existencia          | 15                                 |             | t
 activo              | true                               |             | t
 fecha_actualizacion | "2026-10-29T02:29:42.162888+00:00" |             | t
(6 rows)
```

Pruebas de Rendimiento (Volumen Creciente de Registros Auditados)

- ***Objetivo***: Medir el impacto en el tiempo de respuesta (overhead) y el comportamiento del motor al someter las tablas con *triggers* de auditoría a transacciones iterativas y masivas, garantizando la escalabilidad.
- **Condicionantes:**

La base de datos debe tener los *triggers* activos. Se utilizarán variables de configuración y captura de tiempo nativo de PostgreSQL (clock_timestamp()) para evaluar la duración de las transacciones.

Pasos de Ejecución:

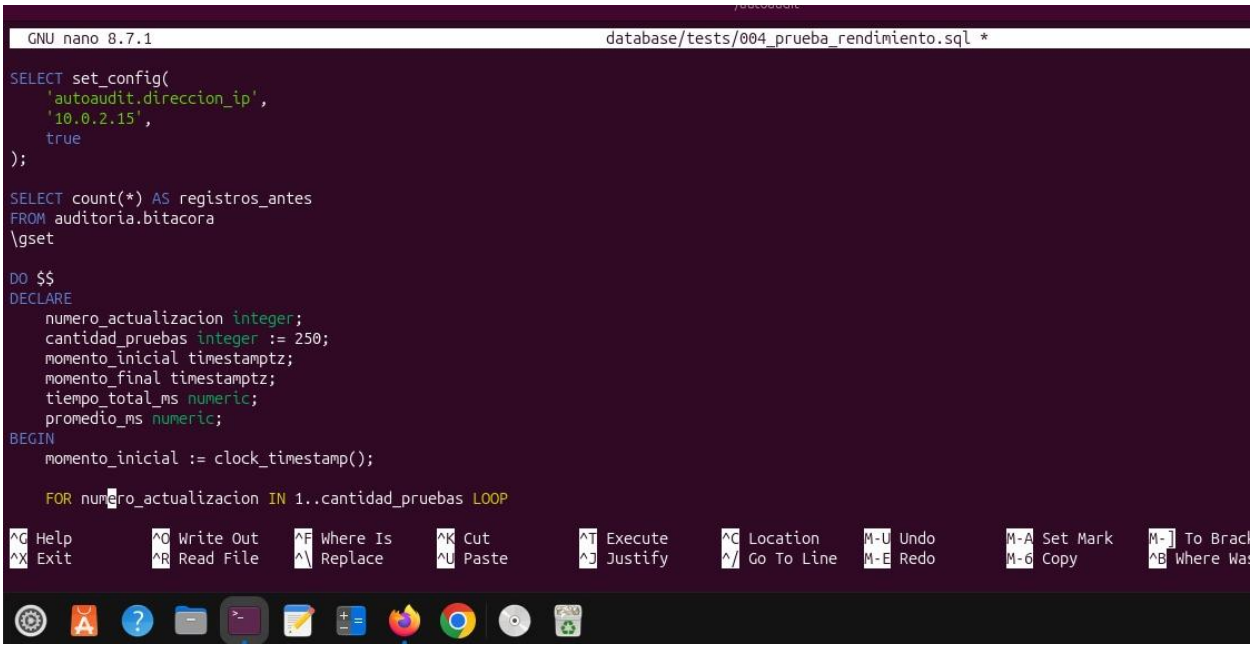
Medición del tiempo de procesamiento mediante un bloque anónimo (PL/pgSQL) que ejecuta iteraciones controladas, registrando el momento inicial y final para calcular el tiempo total y el promedio en milisegundos por transacción

Validación:

- La penalización en el tiempo de procesamiento total (*overhead*) se mantiene en márgenes aceptables respecto a operaciones sin auditoría.
- No se presentan interrupciones por bloqueos cruzados (*deadlocks*) a nivel de base de datos durante el estrés de transacciones.

- La base de datos procesa e inserta correctamente todo el volumen de históricos en formato JSONB.

Evidencia: Preparación del script de prueba de rendimiento estructurando un bloque anónimo para medir con precisión el tiempo de ejecución iterativo.



9. Manual de Usuario

9.1. Acceso y panel

Inicie el servidor y abra http://127.0.0.1:5000. El panel principal resume tablas disponibles, tablas registradas, configuraciones activas, triggers activos y cambios recientes. Las opciones Configuración y Bitácora requieren autenticación administrativa.

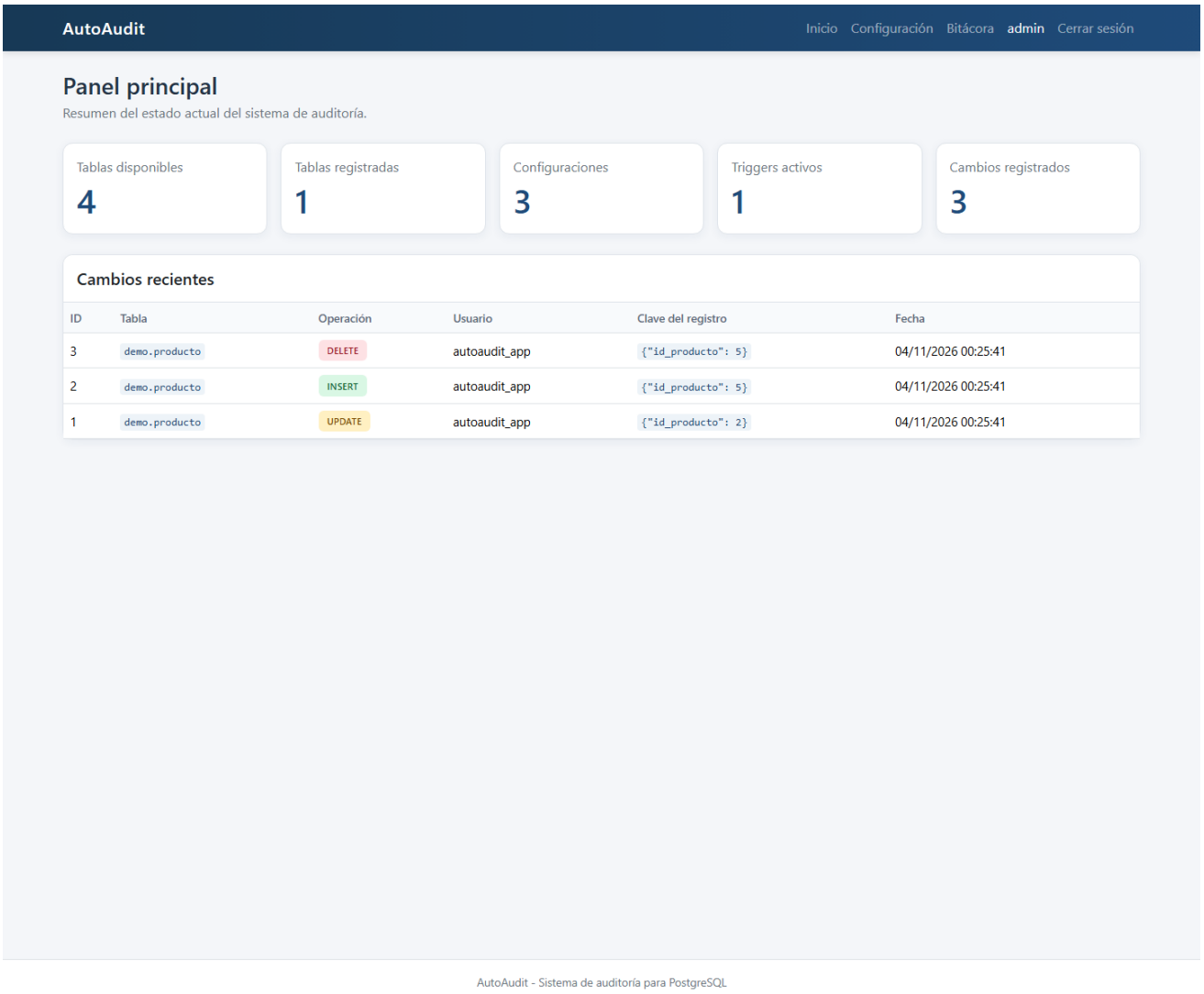


Figura 1. Captura real del panel principal con tres cambios registrados en la bitácora.

9.2. Iniciar/cerrar sesión

Seleccione Iniciar sesión e introduzca el usuario definido en ADMIN_USERNAME y su contraseña original. El hash configurado en .env no se introduce en el formulario. Cierre la sesión con el botón correspondiente al terminar. La sesión tiene una duración configurada de dos horas; no comparta la cuenta administrativa.

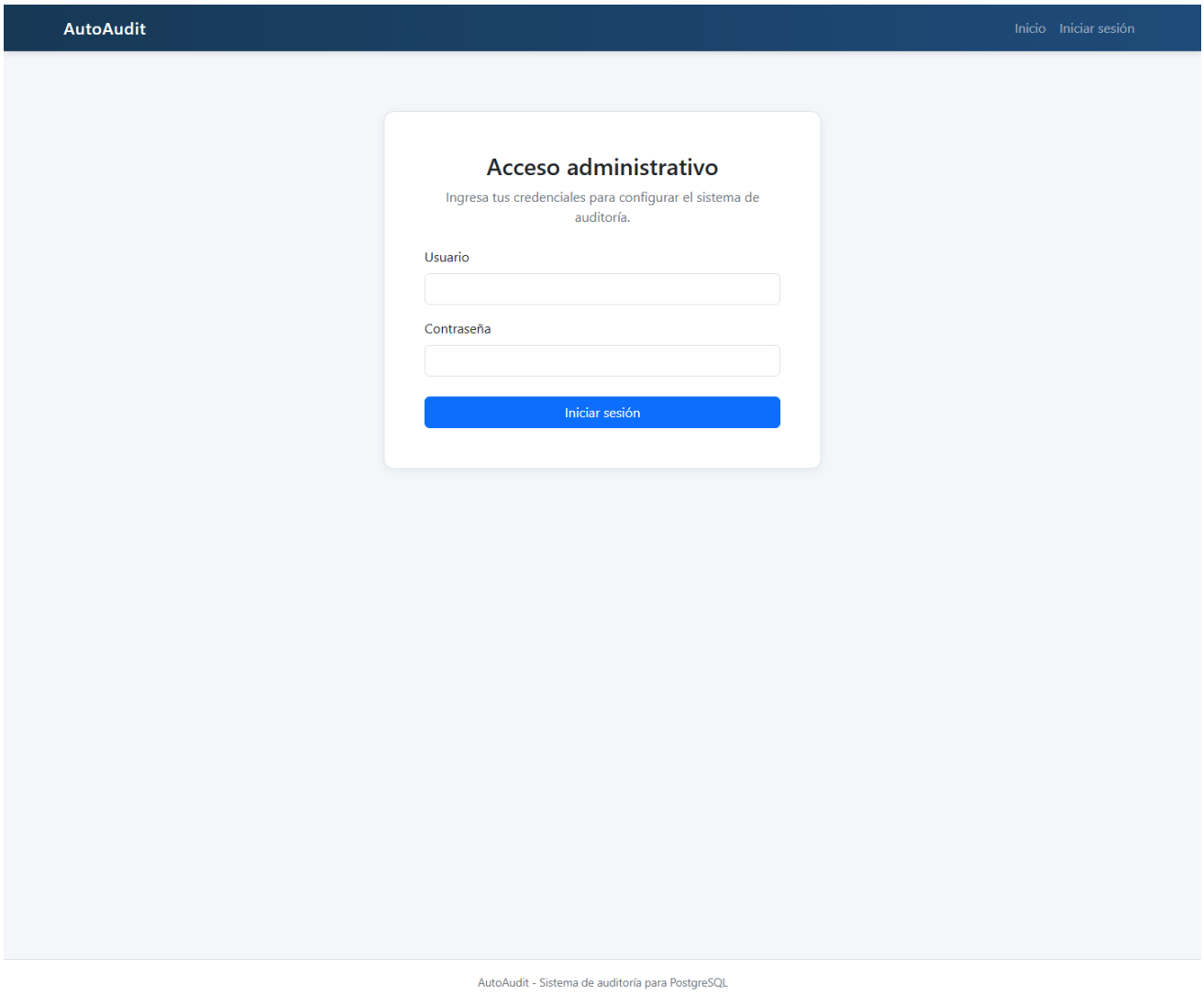


Figura 2. Captura real de la pantalla de acceso administrativo de AutoAudit.

9.3. Configurar la auditoría

1) Entre en Configuración y seleccione Configurar junto a la tabla deseada. Revise esquema, columnas, tipos, clave primaria y el estado del trigger.

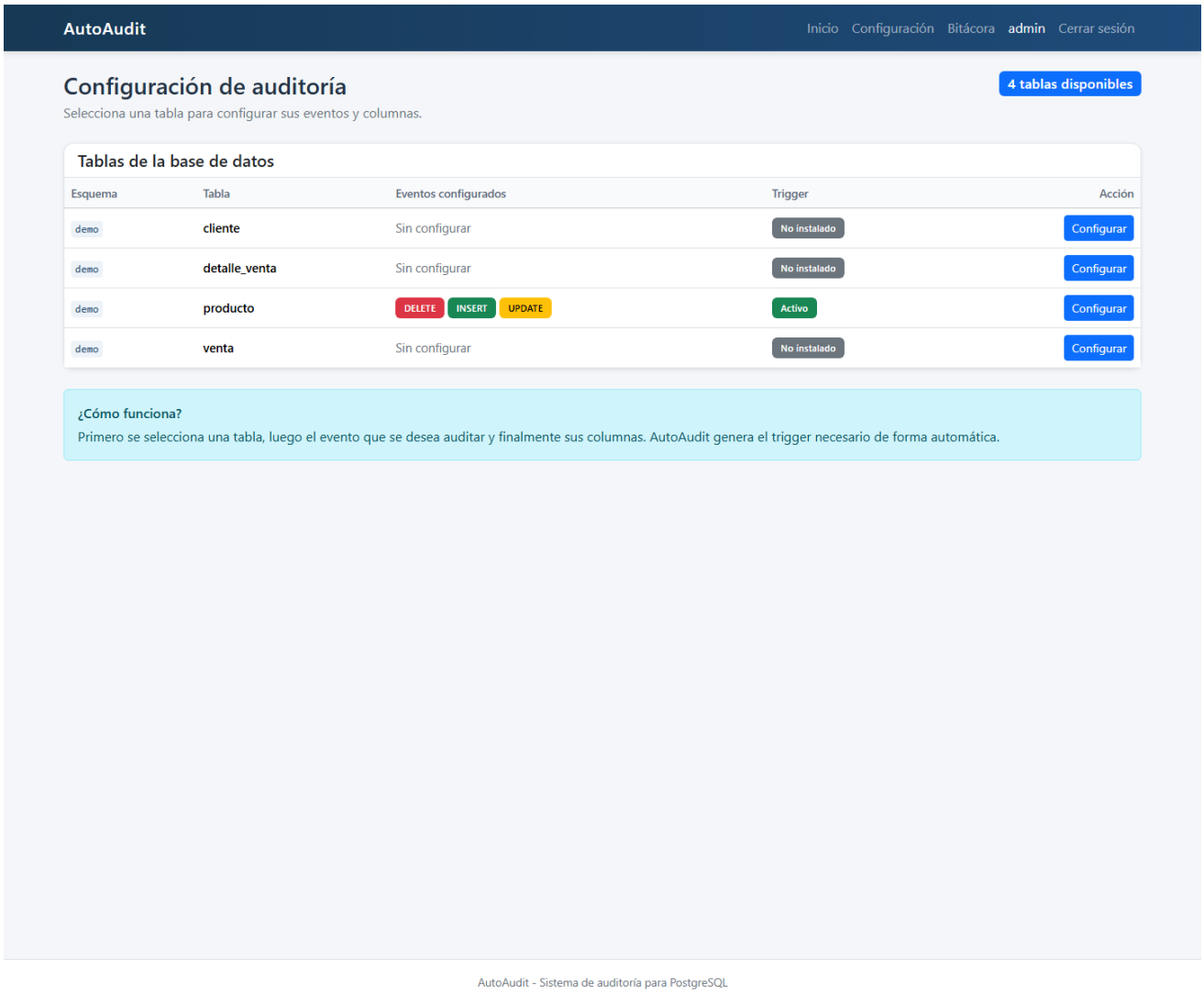


Figura 3. Captura real del listado de tablas, eventos y estado de los triggers.

2) Elija el evento INSERT, UPDATE o DELETE. Marque las columnas de interés. Para UPDATE conviene escoger los campos necesarios; si no se selecciona ninguna columna, la aplicación envía una lista vacía y el procedimiento recibe NULL, que la interfaz trata como auditoría de todas las columnas.

3) Presione Activar/Actualizar para guardar. El sistema guarda una nueva versión de configuración y regenera el trigger de la tabla. Compruebe el mensaje y el estado actualizado.

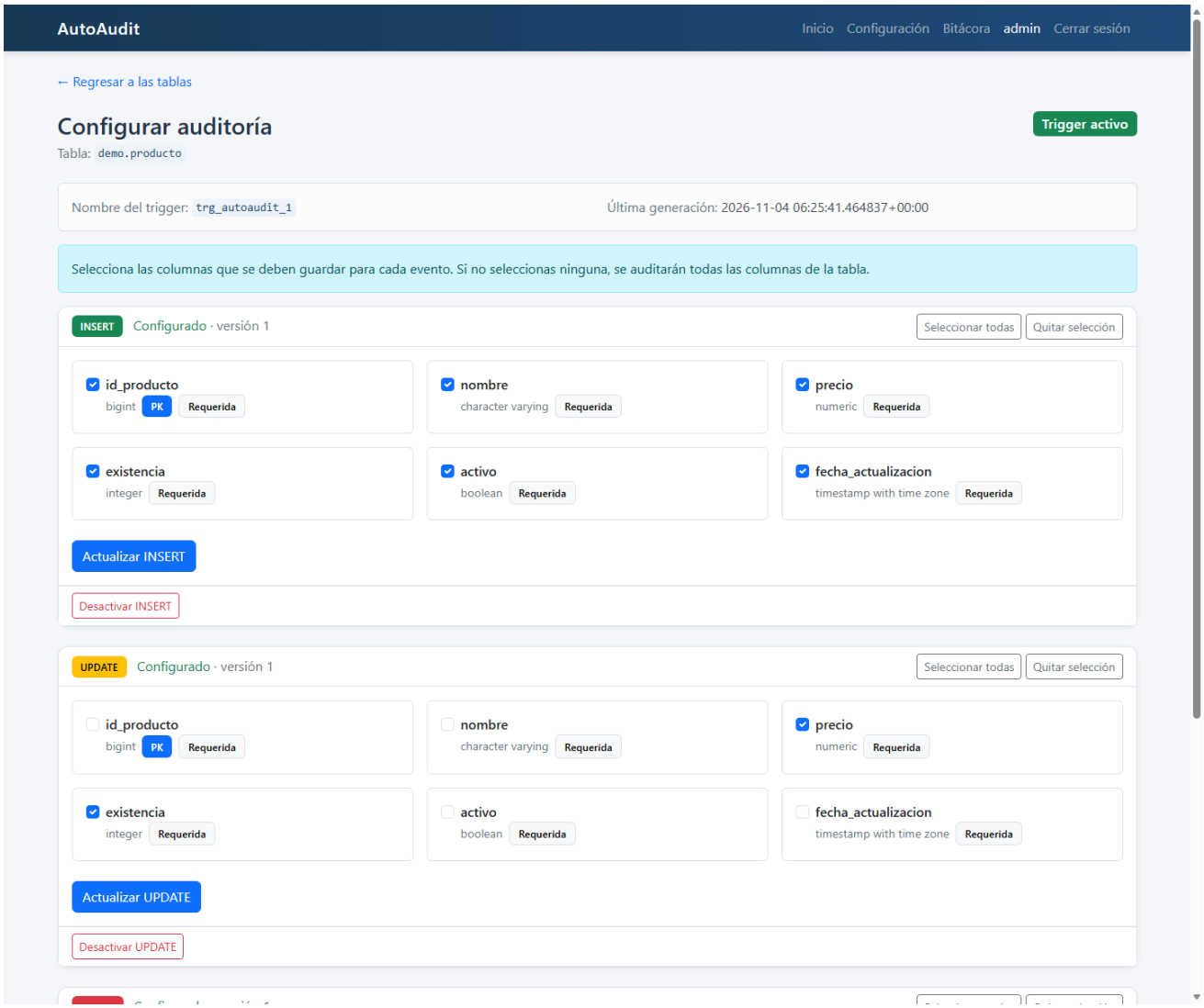


Figura 4. Captura real de la configuración de auditoría de demo.producto y sus columnas.

4) Para detener un evento, use Desactivar en su sección. Esta acción no borra los registros históricos ya guardados.

9.4. Consultar y filtrar la bitácora

Abra Bitácora para revisar los eventos más recientes; la lista se pagina en bloques de 25. Puede filtrar por usuario, esquema, tabla, columna, operación y fechas inicial/final. Seleccione Buscar para aplicar y Limpiar para reiniciar filtros. Ver detalle muestra transacción, configuración, clave del registro, identidad y datos anteriores/nuevos con cambios por columna.

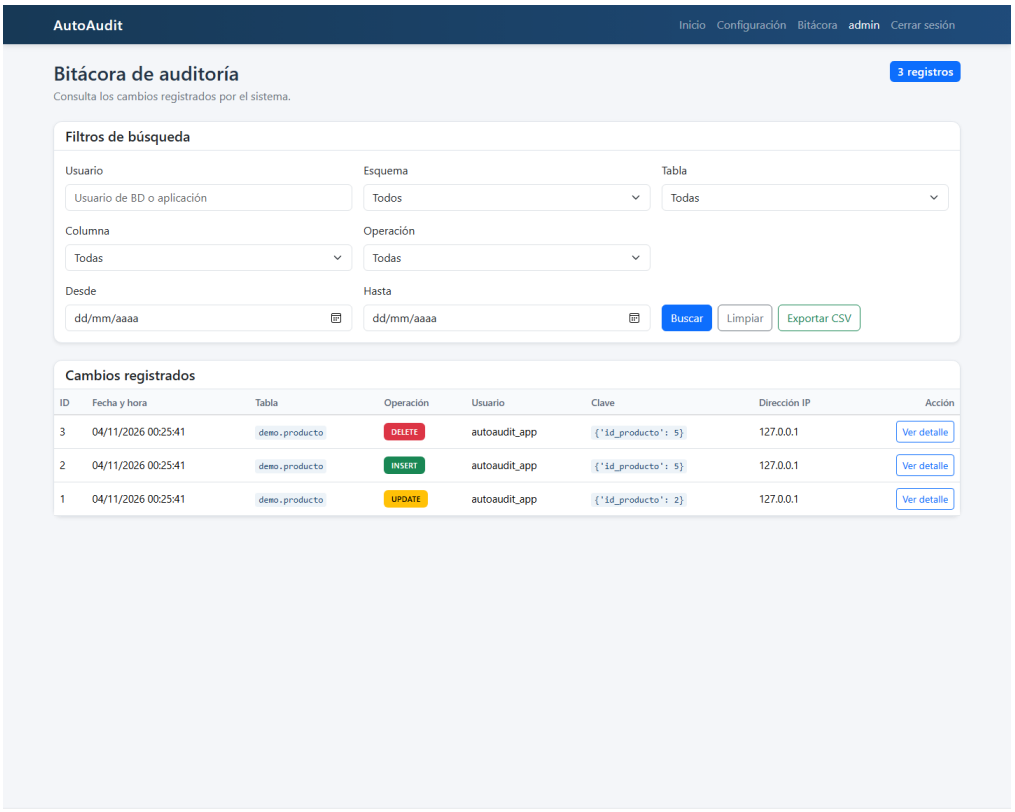


Figura 5. Captura real de la bitácora, con los eventos INSERT, UPDATE y DELETE.

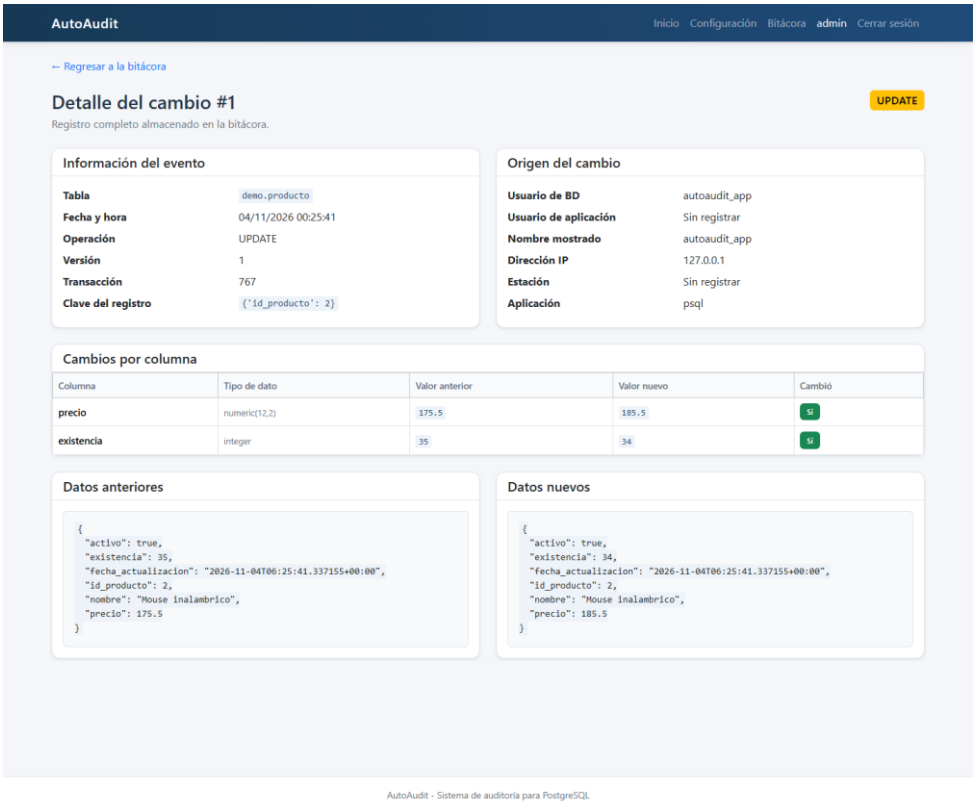


Figura 6. Captura real del detalle del evento UPDATE con sus valores anteriores y nuevos.

9.5. Exportar y comprender eventos

Use Exportar CSV después de definir los filtros. La exportación respeta los filtros y descarga bitacora_autoaudit.csv para abrirlo en una hoja de cálculo. El CSV contiene metadatos del evento y valores de filas; trátelo como información sensible, almacénelo con controles adecuados y compártalo solo con personas autorizadas.

Operación	Interpretación del detalle
INSERT	Datos anteriores vacíos; datos nuevos con la fila creada.
UPDATE	Valores anteriores y nuevos; el detalle identifica columnas modificadas.
DELETE	Datos anteriores con la fila eliminada; datos nuevos vacíos.

9.6. Flujo recomendado

Iniciar sesión → revisar el panel → configurar tabla/evento/columnas → ejecutar un cambio controlado → buscarlo en Bitácora → verificar el detalle → exportar solo si es necesario → cerrar sesión.

10. Conclusiones

- AutoAudit implementa el flujo principal previsto: descubrimiento de tablas, configuración de auditoría por operación y columna, generación de triggers, conservación de versiones, registro centralizado y consulta/exportación del historial. Los casos documentados de INSERT, UPDATE y DELETE y los permisos limitados del rol de aplicación respaldan la viabilidad funcional del sistema en el entorno de pruebas informado.

- Las pruebas de seguridad reportadas muestran controles útiles de privilegio mínimo: la aplicación consulta la bitácora, mientras las escrituras directas, la creación de objetos no permitida y la invocación directa de la función interna se rechazan. La autenticación con hash y la protección CSRF también forman parte de la implementación descrita.

- No obstante, la evidencia disponible es acotada. Los resultados documentados no equivalen a una auditoría independiente ni prueban resistencia frente a todos los ataques. Antes de usar AutoAudit con datos reales o publicarlo, se recomienda desactivar debug, servirlo con WSGI detrás de HTTPS, activar cookies Secure, restringir red y /health, proteger secretos y credenciales privilegiadas, agregar controles contra intentos repetidos de login, actualizar dependencias y establecer copias de seguridad/restauración y monitoreo.

- En conclusión, el proyecto es adecuado como sistema funcional y demostrativo para auditar cambios en PostgreSQL dentro de un entorno controlado. La promoción a producción debe condicionarse a completar las medidas de despliegue, validar los permisos en la instalación destino y ampliar las pruebas de seguridad y operación.